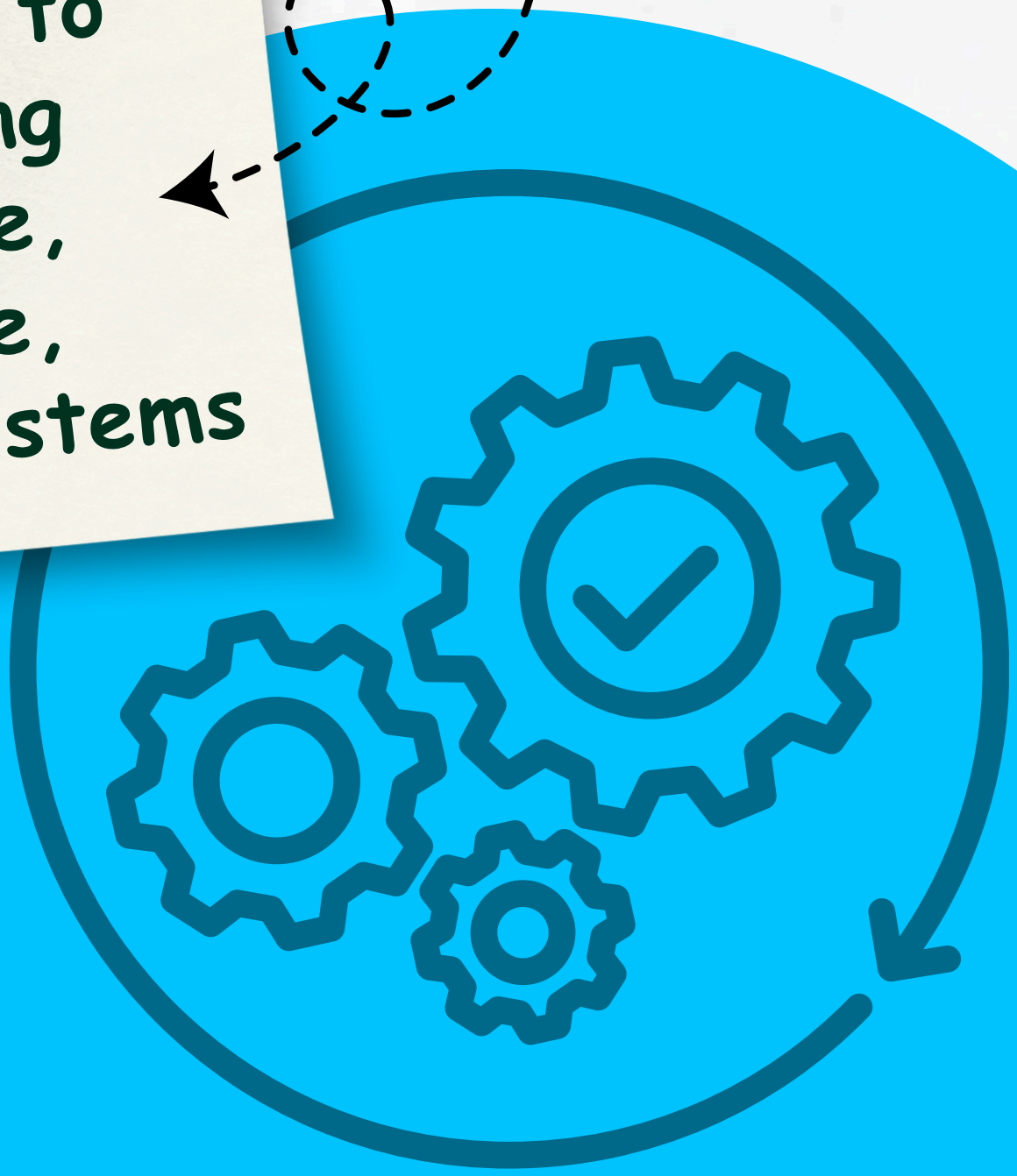
Shalini Goyal

Swipe

System Design

From Beginner to Architect

A 12-Part Roadmap to Designing Scalable, Reliable, Modern Systems

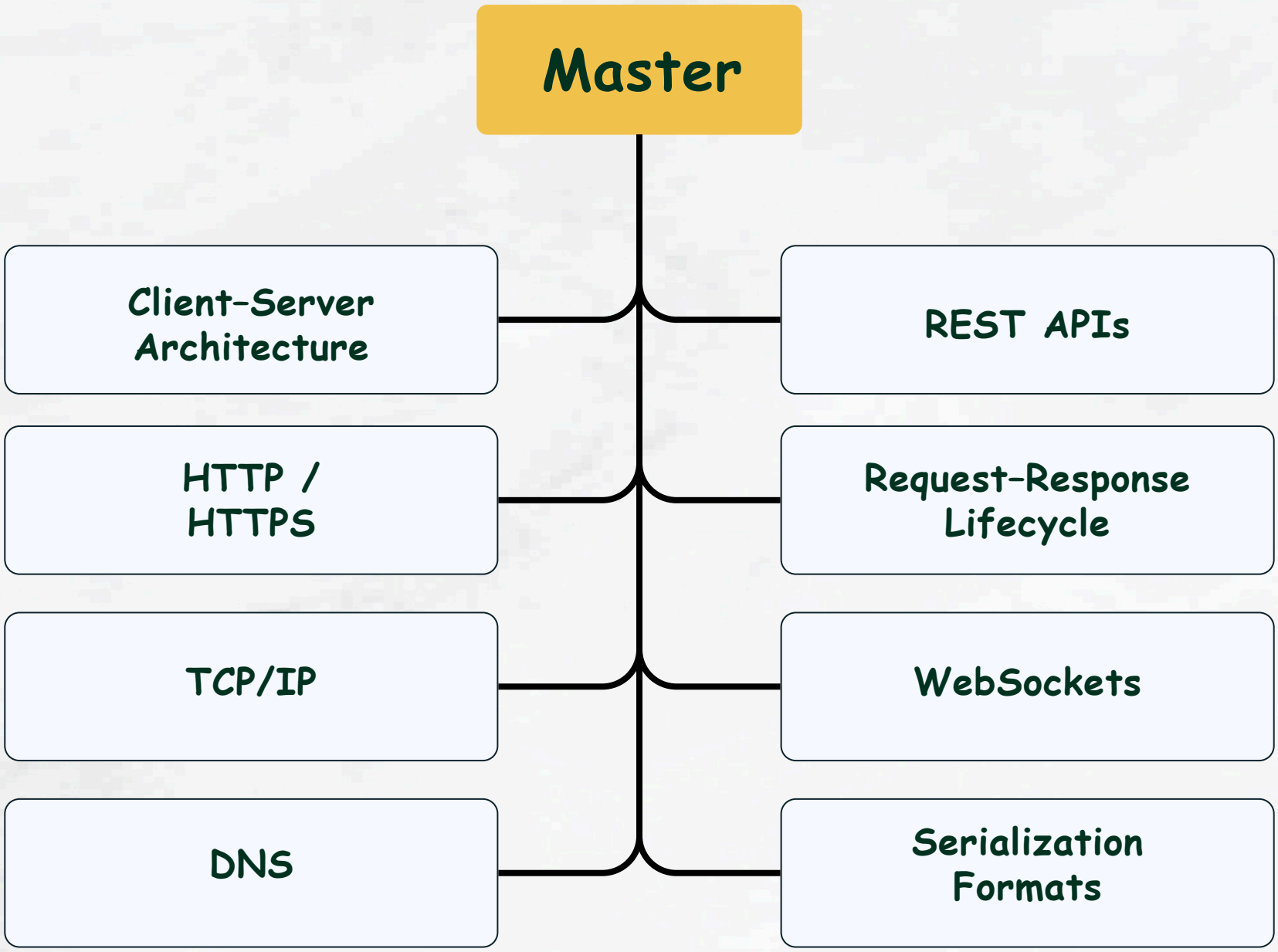


Shalini Goyal

Swipe

Build the Foundations

System design starts with understanding how applications communicate across networks. Before thinking about scale, you should know what happens between a user making a request and a server returning a response.



You should understand

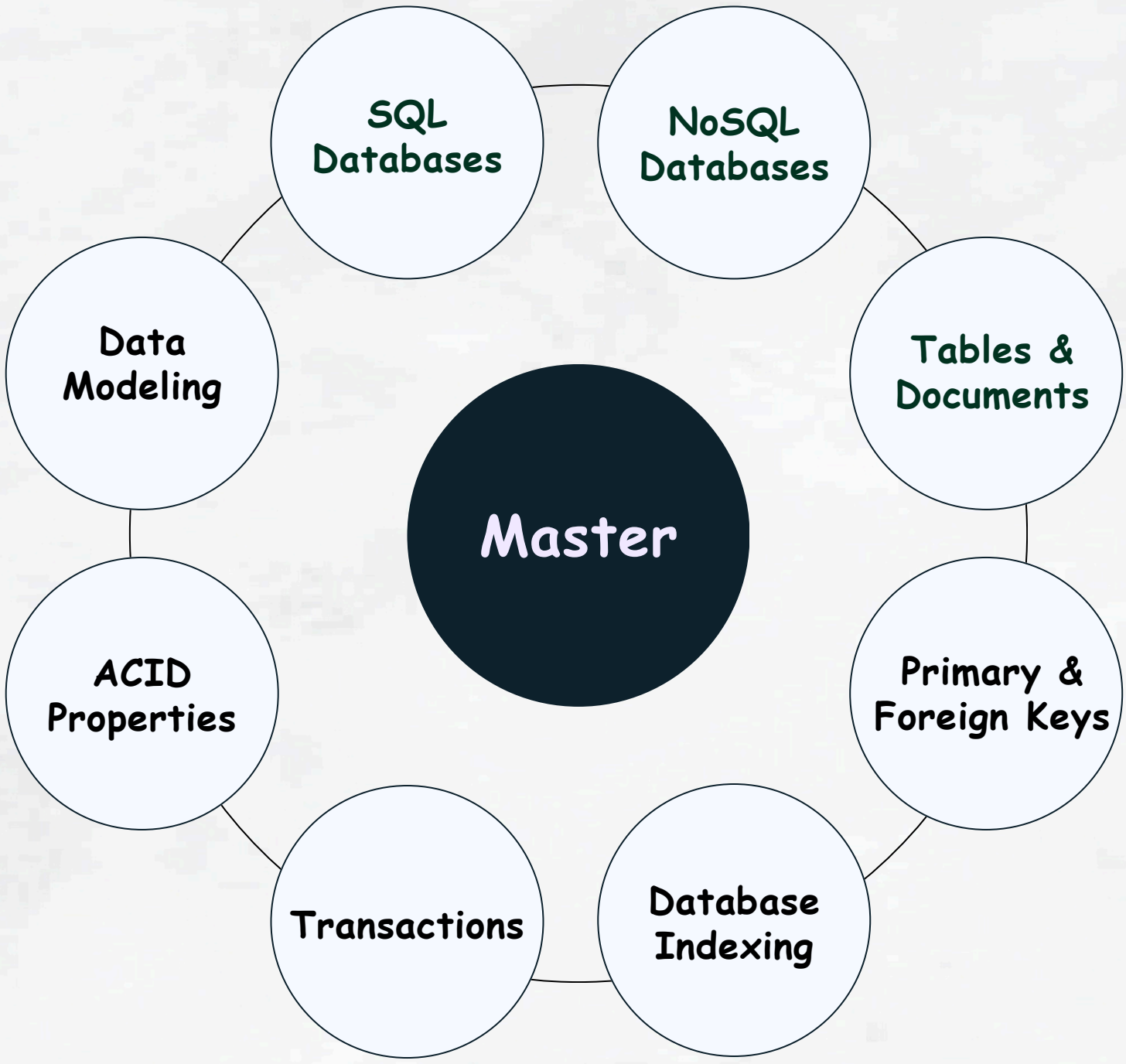
How applications communicate, exchange data, and serve users reliably.

Shalini Goyal

Swipe

Understand Databases & Data Modeling

Architecture decisions are heavily influenced by how your application stores and accesses data. Learn how different database models behave before choosing technology.



You should understand

How to choose storage based on relationships, workload, and consistency.

Shalini Goyal

Swipe

Master Caching

Caching reduces the amount of repeated work your system performs. Used correctly, it improves response times while reducing pressure on databases and backend services.

Master

- Browser Caching
- CDN Caching
- Application Caching
- Redis / Memcached
- Cache-Aside
- Write-Through
- Write-Back
- TTL & Cache Invalidation

You should understand

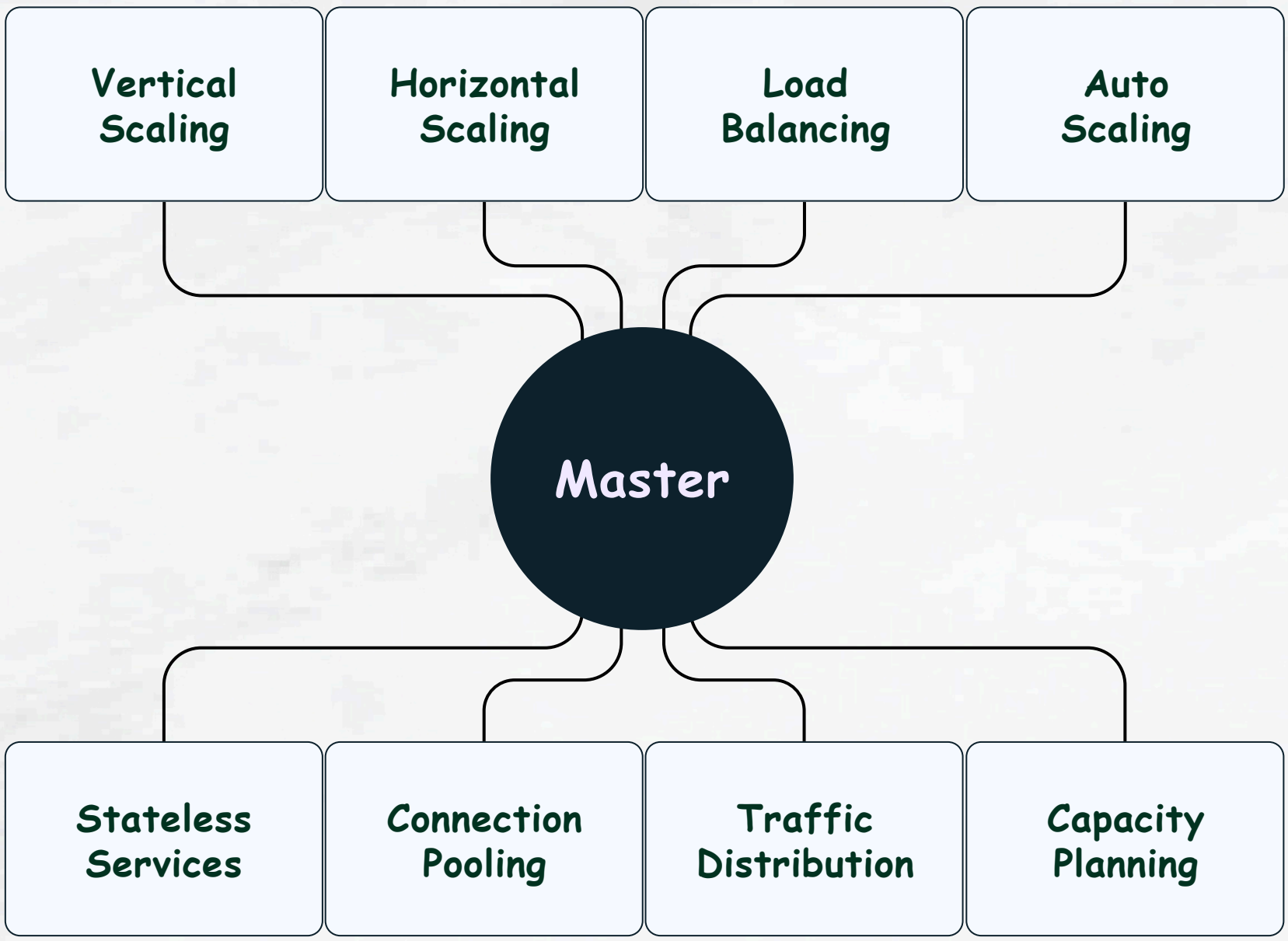
What to cache, where to cache it, and when.

Shalini Goyal

Swipe

Learn How Applications Scale

A system working for 1,000 users may fail completely at 10 million. Scaling is about increasing capacity without redesigning the application every time traffic grows.



You should understand

How to increase system capacity while avoiding single-server bottlenecks.

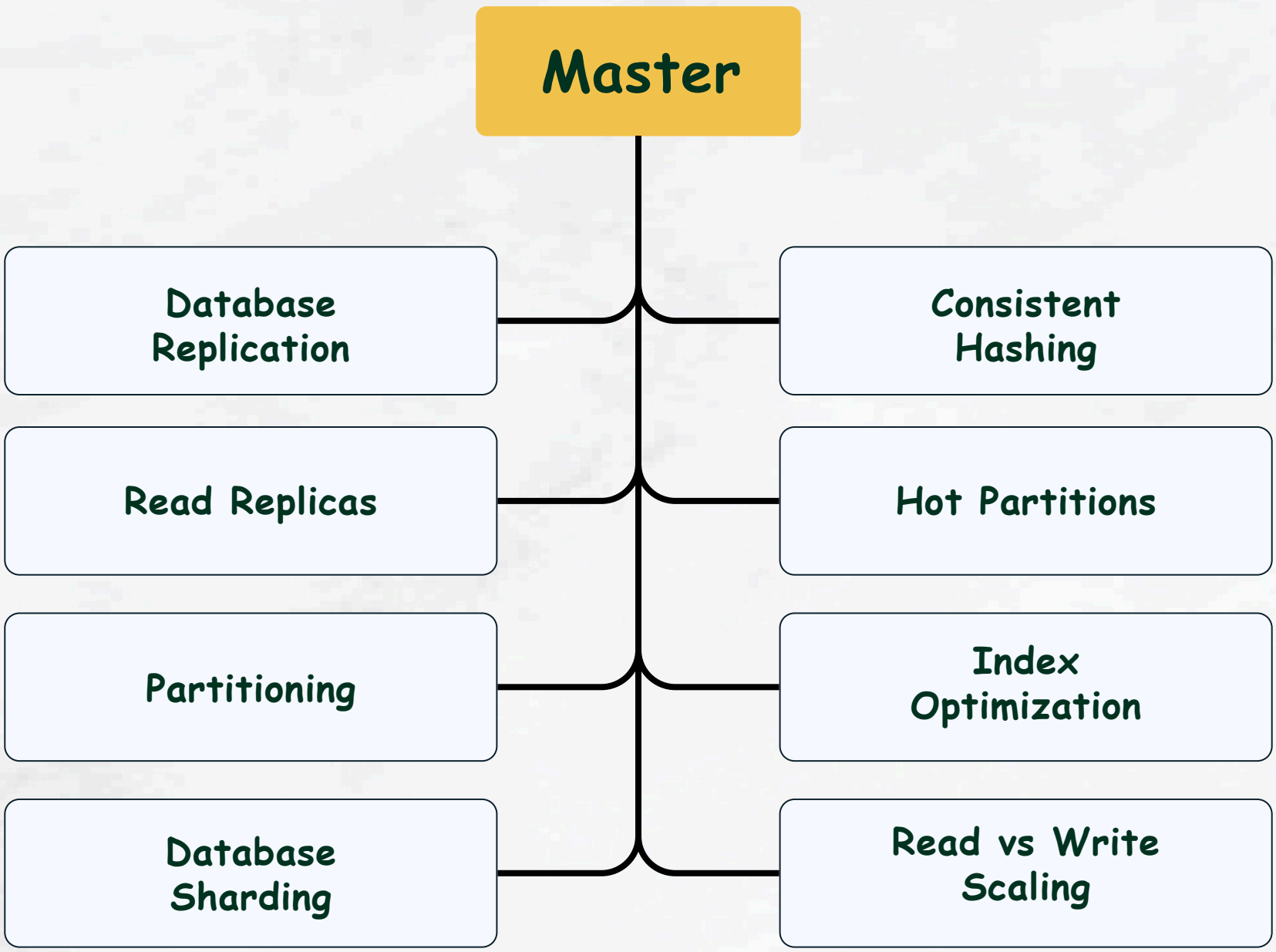


Shalini Goyal

Swipe

Learn Database Scaling

Databases frequently become the first major bottleneck in growing systems. Architects must distribute reads, writes, and datasets without destroying consistency or performance.



You should understand

How databases evolve when one machine can no longer cope.

Shalini Goyal

Swipe

Understand Asynchronous Processing

Users should not wait for every background task to finish. Messaging systems allow applications to move expensive or delayed work outside the synchronous request path.



You should understand

How asynchronous communication improves scalability, responsiveness, and service independence.

Shalini Goyal

Swipe

Enter Distributed Systems

Once your application runs across multiple machines, new problems appear. Machines fail, networks become unreliable, and different nodes may temporarily disagree about state.

Master

- CAP Theorem
- PACELC
- Consistency Models
- Leader Election
- Consensus
- Distributed Locks
- Idempotency
- Distributed Transactions
- Clock & Network Failures

You should understand

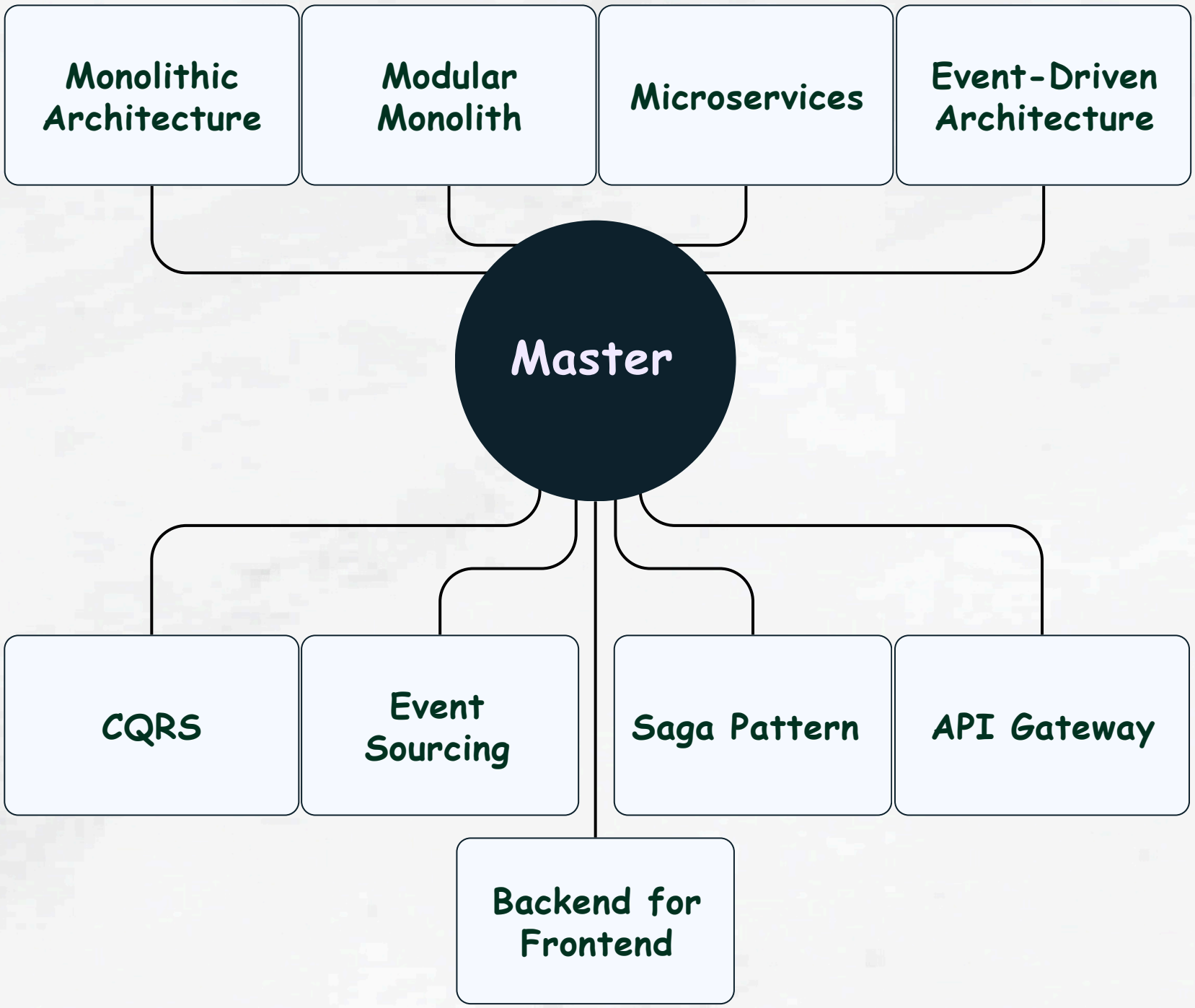
Why distributed architecture is fundamentally about managing trade-offs and failures.

Shalini Goyal

Swipe

Learn Architecture Patterns

There is no single architecture that works for every application. Architects select patterns based on team size, domain complexity, scalability requirements, and operational cost.



You should understand

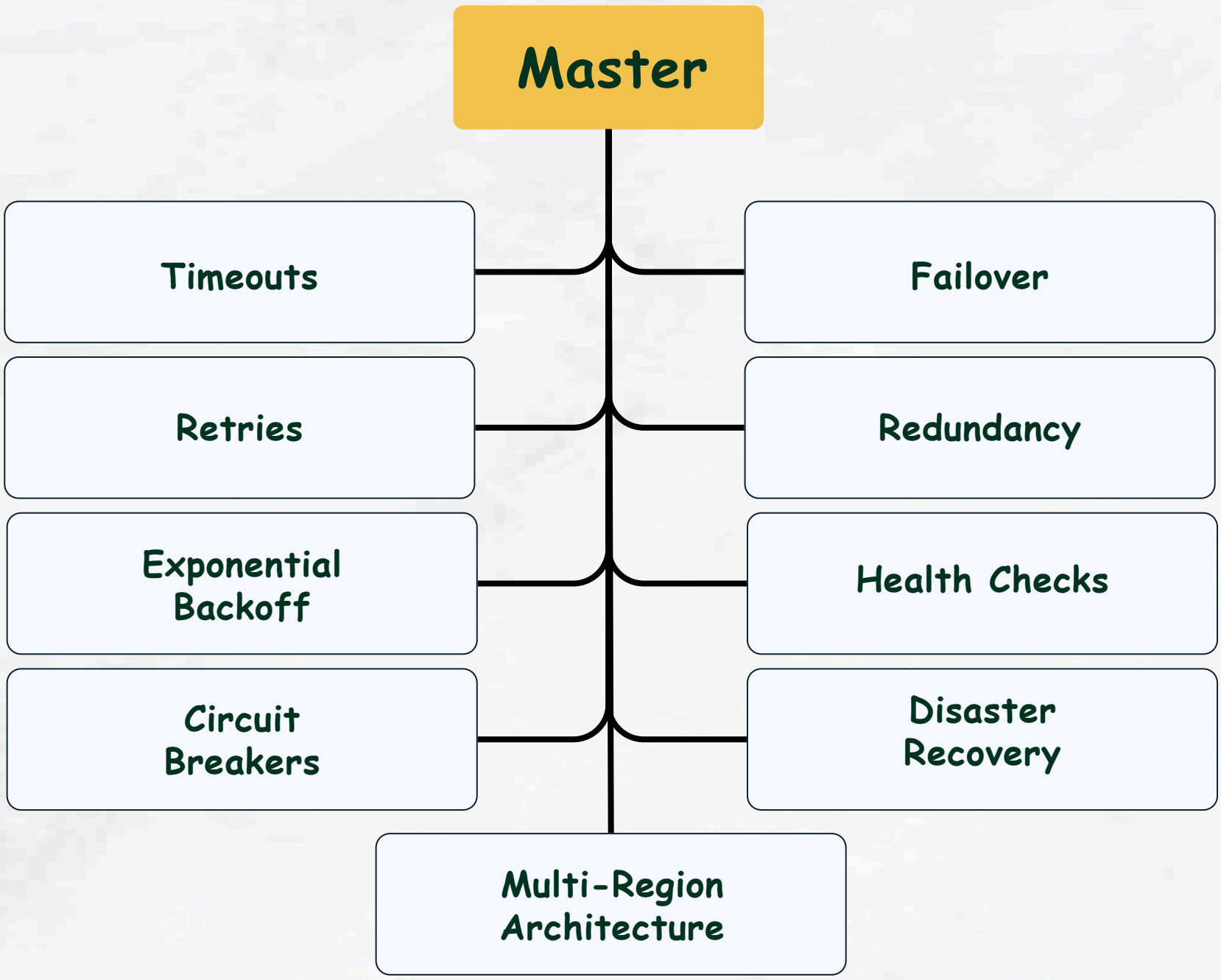
When architectural complexity solves problems and when it creates them.

Shalini Goyal

Swipe

Design for Reliability & Failure

Production architecture assumes that databases, networks, APIs, and services will eventually fail. Good systems contain those failures instead of allowing them to spread.



You should understand

How systems recover gracefully instead of turning failures into outages.


Shalini Goyal

Swipe

Add Security & Observability

A system is incomplete if you cannot secure it or understand what it is doing. Architects design visibility and protection into the architecture from the beginning.

Security	Observability
Authentication	Logs
Authorization	Metrics
OAuth / OIDC	Distributed Tracing
JWT	APM
TLS Encryption	Alerting
Secrets Management	SLOs / SLIs
Rate Limiting	Dashboards
Zero Trust	

You should understand

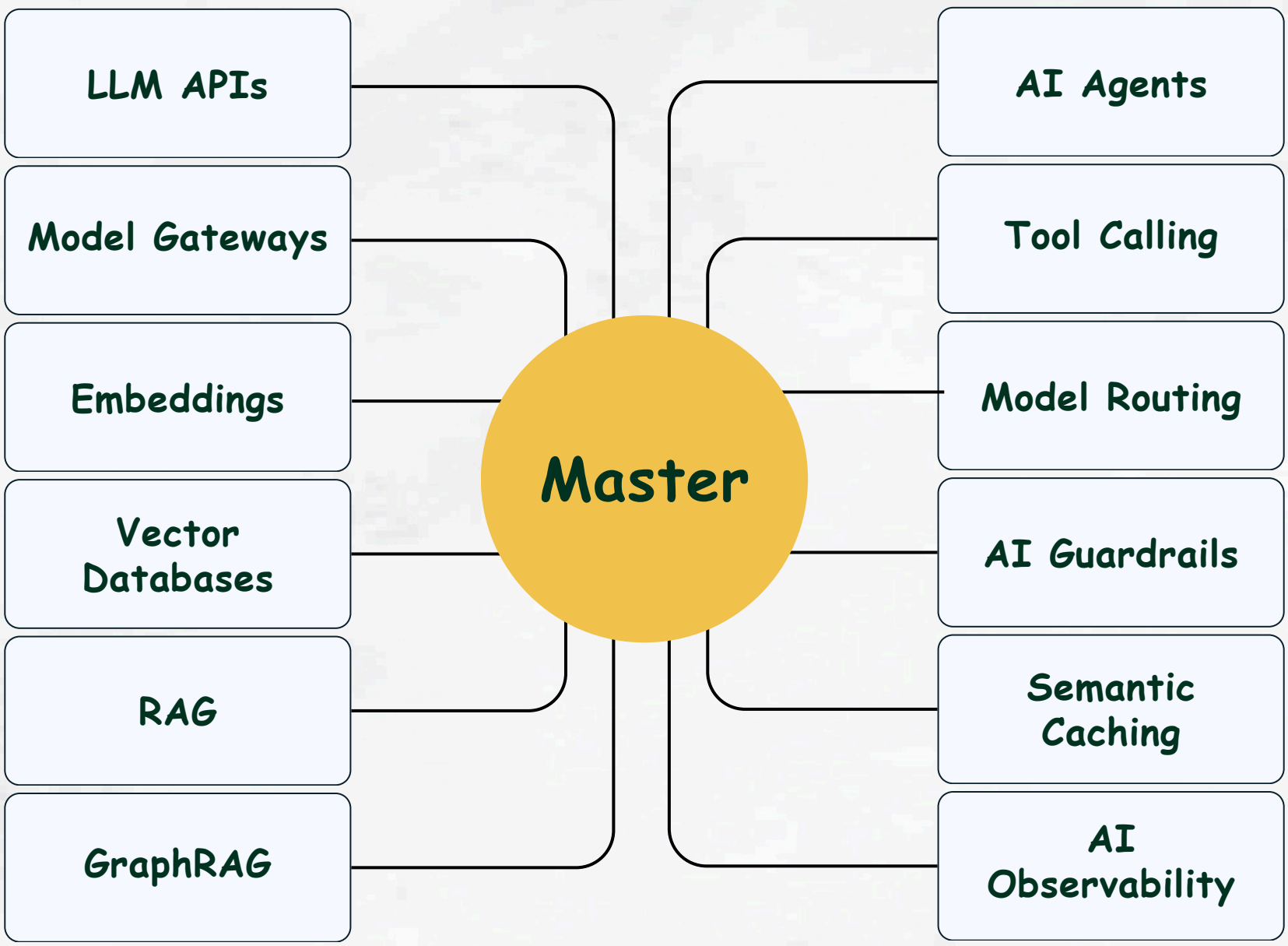
How to protect systems while detecting problems before users do.

Shalini Goyal

Swipe

Learn AI-Native System Design

Modern applications increasingly combine traditional software with models, retrieval systems, and agents. These systems introduce new challenges around latency, cost, context, security, and unpredictable outputs.



You should understand

How AI becomes one reliable component inside a larger architecture.



Shalini Goyal

Swipe

Think Like an Architect

Knowing technologies is only the beginning. Architects translate business requirements into technical decisions while constantly balancing reliability, performance, complexity, cost, and future growth.

Before designing, ask

- What problem are we solving?
- Who are the users?
- How much traffic do we expect?
- What data must we store?
- What consistency is required?
- Where are the bottlenecks?
- What happens when components fail?
- What security risks exist?
- How will we monitor everything?
- What will this architecture cost?
- How will it evolve in three years?

Beginner	Engineer	Senior Engineer	Architect
Understands individual components and basic architecture concepts.	Connects components into functional end-to-end systems.	Evaluates bottlenecks, failures, scale, and technical trade-offs.	Designs systems around business goals, constraints, cost, and evolution.

System design mastery is not memorizing diagrams.

It is learning how to make the right trade-off for the right system at the right scale.



Shalini Goyal

Follow for more